

ECS401: Unit 4 Lab Progress Test

FEEDBACK: Answers and Explanations

Work hard and you will eventually be able to program.

If you take it seriously, this sheet will help improve your understanding and the lab tutors and peer tutors can give you more help if you need it.

1. **Complete the dry run table** and then **give** the final integer value of `z` printed to the screen.

```
int z = 0;                // L1
for(int j = 1; j <= 2; j++) // L2
{
    z = z + 1;            // L3
}
System.out.println(z);    // L4
```

DRY RUN TABLE:

Line	z	j	j <= 2 ?	printed to screen
L1	0			
L2		1	1 <= 2 ? true	
L3	1			
L2		2	2 <= 2 ? true	
L3	2			
L2		3	3 <= 2 ? false	
L4				2

CORRECT ANSWER:

2

EXPLANATION:

The for loop adds one to `z` each time its body executes. The loop counter `j` starts at 1 and stops when no longer less than or equal to 2. It therefore counts 1, 2 and stops as 3 is not less than or equal to 2. That means `z` is incremented (ie one added to it) twice, once when `j` is 1 and then when `j` is 2. So overall 2 is added to the original value of `z` which was 0 giving answer 2.

2. **Complete the dry run table** and then **give** the final integer value of z printed to the screen.

```
int z = 5;                // L1
for(int j = 1; j <= 4; j++) // L2
{
    z = z + 1;            // L3
}
System.out.println(z);    // L4
```

DRY RUN TABLE:

Line	z	j	j <= 4 ?	printed to screen
L1	5			
L2		1	1 <= 4 ? true	
L3	6			
L2		2	2 <= 4 ? true	
L3	7			
L2		3	3 <= 4 ? true	
L3	8			
L2		4	4 <= 4 ? true	
L3	9			
L2		5	5 <= 4 ? false	
L4				9

CORRECT ANSWER:

9

EXPLANATION:

This time z starts at 5, so it will be similar to the above but we will be adding to 5 not 0 and going round the loop more times as its test is different.

j starts at 1, which is <=4, so we add 1 to the 5 in z to give 6

j becomes 2, which is <=4, so we add 1 to the 6 in z to give 7

j becomes 3, which is <=4, so we add 1 to the 7 in z to give 8

j becomes 4, which is <=4, so we add 1 to the 8 in z to give 9

j becomes 5, which is NOT <=4, so we leave the loop and z has 9 stored in it from the last step.

3. **Complete the dry run table** and then **give** the final integer value of z printed to the screen.

```
int z = 1;                // L1
for(int k = 1; k < 1; k++) // L2
{
    z = z + 1;             // L3
}
System.out.println(z);    // L4
```

DRY RUN TABLE:

Line	z	k	k < 1 ?	printed to screen
L1	1			
L2		1	1 < 1 ? false	
L4				1

CORRECT ANSWER:

1

EXPLANATION:

z starts with 1. The loop test says to enter the loop body while k is less than 1. But k starts at 1 so is never less than 1. The body of the loop is not entered and z has its original value of 1.

4. **Complete the dry run table** and then **give** the final integer value of z printed to the screen.

```
int z = 0;                // L1
for(int k = 1; k < 4; k++) // L2
{
    z = z + 2;             // L3
}
System.out.println(z);    // L4
```

DRY RUN TABLE:

Line	z	k	k < 4 ?	printed to screen
L1	0			
L2		1	1 < 4 ? true	
L3	2			
L2		2	2 < 4 ? true	
L3	4			
L2		3	3 < 4 ? true	
L3	6			
L2		4	4 < 4 ? false	
L4				6

CORRECT ANSWER:

6

EXPLANATION:

The loop this time adds 2 each time round. Also note that the test is different. It is ($k < 4$). It is asking if k is less than 4 not if it is less than or equal to 4. That means it stops a step earlier than in the first two questions.

So

k starts at 1, which is < 4 , so we add 2 to the 0 in z to give 2

k becomes 2, which is < 4 , so we add 2 to the 2 in z to give 4

k becomes 3, which is < 4 , so we add 2 to the 4 in z to give 6

k becomes 4, which is NOT < 4 , so we leave the loop and z has 6 stored in it from the last step.

5. **Complete the dry run table** and then **give** the final integer value of z printed to the screen.

```

int z = 0;                                // L1
for(int j = 1; j <= 2; j++) // L2
{
    z = z + 1;                            // L3
}
for(int k = 1; k <= 2; k++) // L4
{
    z = z + 1;                            // L5
}
System.out.println(z);                    // L6

```

DRY RUN TABLE:

L	z	j	j <= 2 ?	k	k <= 2 ?	printed to screen
L1	0					
L2		1	1 <= 2 ? true			
L3	1					
L2		2	2 <= 2 ? true			
L3	2					
L2		3	3 <= 2 ? false			
L4				1	1 <= 2 ? true	
L5	3					
L4				2	2 <= 2 ? true	
L5	4					
L4				3	3 <= 2 ? false	
L6						4

CORRECT ANSWER:

4

EXPLANATION:

This is one loop following another. The whole of the first loop is executed leaving some new value in z. The second loop then starts and does the same thing again, but adding to the value left by the first loop.

The first loop counts from 1 to 2 so adds 2 on similar to as in question 1 above. That gives a value of 2 in z as it started at 0. The second loop again goes round 2 times so adds 2 on again, leaving a total of $0+2+2 = 4$ in z.

Terminology

6. A **for loop** is a counter-controlled loop: a programming construct that uses a counter to repeat code a fixed number of times. Highlight/underline each of the for loops in the following fragment of code.

```
int z = 0;
for(int j = 1; j <= 4; j++)
{
    z = z + 1;
}
for(int k = 1; k <= 4; k++)
```

```

{
    z = z + 1;
}
System.out.println(z);

```

7. The **loop body** is a block that is repeatedly executed under the control of the loop. Highlight/underline the loop body in the following fragment of code.

```

int z = 0;
for(int j = 1; j <= 4; j++)
{
    z = z + 1;
}

```

8. A **loop counter** is a variable that keeps track of how many times a loop body has executed. Highlight/underline the loop counter in the following fragment of code.

```

int z = 0;
for(int j = 1; j <= 4; j++)
{
    z = z + 1;
}

```

9. A **loop counter** must be **initialised**. Highlight/underline the statement that initialises the loop counter.

```

int z = 0;
for(int j = 1; j <= 4; j++)
{
    z = z + 1;
}

```

10. The termination of a **for loop** is controlled by a boolean expression. Highlight/underline the boolean expression in the following fragment of code.

```
int z = 0;
for(int j = 1; j <= 4; j++)
{
    z = z + 1;
}
```